Trabajo Práctico 2 – Reconstrucción tridimensional estéreo

I308 – Visión Artificial



Universidad de San Andrés

Integrantes: Nicolas Diez, Facundo Tedesco, Alejandro Garcia Giacchetta

Resumen

El trabajo desarrolla un pipeline de reconstrucción tridimensional de un zapato mediante estéreo multivista. A partir de una calibración estéreo con un RMS de reproyección de 0,474 px, se rectificaron las imágenes y se estimó la disparidad mediante StereoSGBM y CREStereo. Las nubes locales se reproyectaron con la matriz Q y se transformaron a un sistema global utilizando poses calculadas a partir de un tablero ChArUco. La reconstrucción definitiva empleó CREStereo sobre 27 capturas con poses válidas y combinó recorte espacial, voxelización, soporte de tres vistas adicionales y limpieza estadística. La nube final contiene 24 037 puntos. Mediante PCA y percentiles se estimaron dimensiones de $283{,}6 \times 92{,}7 \times 114{,}0$ mm. La selección del método fue cualitativa, dado que no se dispone de dimensiones físicas de referencia para evaluar la exactitud métrica.

Introducción y Marco Teórico

La reconstrucción estéreo recupera puntos 3D a partir de sus proyecciones en dos cámaras y permite integrar distintas capturas en un marco común. Su escala y precisión dependen de la calibración, las correspondencias entre imágenes y las poses estimadas [1].

Modelo pinhole y distorsión

El modelo *pinhole* proyecta un punto $\mathbf{X}_c = (X_c, Y_c, Z_c)^T$ de la cámara sobre el píxel (u, v) según

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K\mathbf{X}_c, \qquad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad s = Z_c. \tag{1}$$

Los intrínsecos son las focales (f_x, f_y) en píxeles y el punto principal (c_x, c_y); asumimos oblicuidad nula. Para un punto del mundo, $\mathbf{X}_c = R\mathbf{X}_w + \mathbf{t}$ incorpora la orientación y posición relativas. Las lentes reales introducen distorsión radial (coeficientes k_1, k_2, k_3) y tangencial (p_1, p_2). Calibrar permite estimar K y estos coeficientes para asociar correctamente cada píxel a un rayo de proyección.

Calibración estéreo y error de reproyección

La calibración conjunta estima además la rotación R y traslación $\mathbf{t}$ entre las dos cámaras: $\mathbf{X}_R = R\mathbf{X}_L + \mathbf{t}$. Su módulo $B = \|\mathbf{t}\|$ es la separación física entre centros ópticos. Usamos observaciones simultáneas de un patrón de dimensiones conocidas, desde distintas posiciones, para ajustar estos parámetros minimizando el error entre esquinas detectadas y reproyectadas [2]. El RMS de reproyección es la raíz del promedio de las distancias cuadráticas entre ambas posiciones, expresado en píxeles. Un RMS bajo indica consistencia del ajuste, pero no garantiza por sí solo la precisión tridimensional. El tamaño físico del patrón fija la escala métrica del sistema.

Geometría epipolar y rectificación

Cada punto de una imagen determina una línea epipolar en la otra: sus coordenadas homogéneas satisfacen $\tilde{\mathbf{x}}_R^T F \tilde{\mathbf{x}}_L = 0$, donde F es la matriz fundamental. La rectificación usa la calibración para corregir distorsión y transformar ambas imágenes a planos con líneas epipolares horizontales. Idealmente, los puntos correspondientes quedan en la misma fila ($v_L = v_R$), de modo que la búsqueda de correspondencias pasa de dos dimensiones a una.

Disparidad y profundidad

La disparidad horizontal $d = u_L - u_R$ se estima asociando píxeles de ambas imágenes. En un par rectificado ideal, con focal común f, puntos principales horizontales coincidentes y separación B, la triangulación da

$$d = u_L - u_R, \qquad Z = \frac{fB}{d}. \tag{2}$$

La matriz Q, obtenida en la rectificación, generaliza la reproyección a las tres coordenadas de la cámara izquierda rectificada, incluyendo una eventual diferencia entre puntos principales. Como B está expresado en milímetros, también lo están las coordenadas reconstruidas. La profundidad se vuelve más sensible al error de disparidad cuando crece Z: $|\Delta Z| \approx Z^2|\Delta d|/(fB)$. En regiones ocluidas o poco texturadas las correspondencias son especialmente inciertas; por eso comparamos StereoSGBM, basado en costos y suavidad espacial [3], con CREStereo, que utiliza correlación aprendida y refinamiento recurrente [4].

Metodología

Calibración, disparidad y nube local

Procesamos 28 pares estéreo de 1280×720 píxeles de un zapato sobre un tablero ChArUco. Utilizamos `calib-stereo`, herramienta de la cátedra basada en OpenCV, con 14 pares de un *checkerboard* de 9×6 esquinas internas y cuadrados de 28 mm. Detectamos las esquinas con `findChessboardCorners()` y las refinamos con `cornerSubPix()`; al recalibrar, `stereoCalibrate()` ajustó las matrices intrínsecas, la distorsión y la relación rígida entre cámaras minimizando el error de reproyección. La rectificación obtiene nuevas proyecciones y mapas de remuestreo (operaciones de `stereoRectify()` e `initUndistortRectifyMap()`). Reutilizamos los archivos `stereo_calibration.pkl` y `stereo_maps.pkl` para corregir cada par con `cv2.remap()`, sin volver a calibrar. Verificamos el ajuste con el RMS y el error vertical entre esquinas ChArUco rectificadas [5].

Como referencia probamos StereoSGBM en modo `SGBM_3WAY`: disparidad mínima 0, rango inicial 128, ventana 5×5, unicidad 10 y penalizaciones $P_1 = 8b^2 = 200$, $P_2 = 32b^2 = 800$ ($b = 5$). También ensayamos rangos más amplios y selecciones A–D de capturas; D utilizó 21 vistas, excluidas 4 y 19. Para el resultado definitivo usamos un CREStereo ONNX preentrenado, modo *combined*, 10 iteraciones, a 1280×720 píxeles y con las 27 capturas que tenían una pose válida. No reentrenamos la red.

Cada mapa de disparidad se convirtió en puntos de la cámara izquierda rectificada con `cv2.reprojectImageTo3D()` y la matriz Q, conservando sus colores. Descartamos profundidades no positivas o superiores a 1500 mm.

Pose y transformación al marco global

Detectamos las esquinas de un ChArUco de 9×13 cuadrados de 42 mm, marcadores de 31,5 mm y diccionario `DICT_5X5_100`. A partir de estas correspondencias, `solvePnP()` estimó el vector de rotación $\mathbf{r}_{cb}$ y la traslación $\mathbf{t}_{cb}$ para cada captura. La representación de Rodrigues codifica una rotación como $\mathbf{r} = \theta\hat{\mathbf{u}}$: su dirección indica el eje de giro y su norma, el ángulo θ en radianes. Mediante `cv2.Rodrigues()` obtenemos la matriz R_{cb} de 3×3, que luego usamos en las transformaciones rígidas [6]. PnP transforma puntos del tablero b a la cámara izquierda *original* c mediante $\mathbf{X}_c = R_{cb}\mathbf{X}_b + \mathbf{t}_{cb}$. En coordenadas homogéneas es exactamente equivalente a

$$\widetilde{\mathbf{X}}_c = T_{c,b}\widetilde{\mathbf{X}}_b, \qquad T_{c,b} = \begin{bmatrix} R_{cb} & \mathbf{t}_{cb} \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad \widetilde{\mathbf{X}} = \begin{bmatrix} \mathbf{X} \\ 1 \end{bmatrix}. \tag{3}$$

Esta representación 4×4 permite componer rotaciones y traslaciones en un único producto. Como la nube local pertenece a la cámara *rectificada* r, deshacemos primero su rotación R_1, invertimos después PnP y finalmente pasamos al marco global w:

$$\boxed{T_{w,r} = T_{w,b}\, T_{c,b}^{-1} \begin{bmatrix} R_1^T & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix}}, \qquad \widetilde{\mathbf{X}}_w = T_{w,r} \widetilde{\mathbf{X}}_r. \tag{4}$$

La transformación fija $T_{w,b}$ invierte los ejes Y y Z del tablero y traslada el origen: el plano queda en $Z_w = 0$ y el zapato del lado positivo. Así expresamos todas las nubes y poses en milímetros dentro del mismo marco.

Recorte, fusión y limpieza

Con Open3D aplicamos una ROI global alrededor del tablero (margen lateral de 42 mm, hasta 336 mm por encima y 42 mm por debajo); luego voxelizamos cada vista a 3 mm. Una ROI más ajustada selecciona el zapato: $X_w \in [130, 270]$, $Y_w \in [101, 436]$ y $Z_w \in [12, 336]$ mm. Este último umbral reduce los puntos del tablero, aunque puede quitar parte de la suela. Los límites fueron definidos manualmente a partir de la escena, no estimados automáticamente.

Los filtros clásicos de densidad y DBSCAN no eliminaron todo el ruido observado con SGBM. Por eso incorporamos **soporte multivista**: un punto $\mathbf{p}$ reconstruido en la vista i se conserva solo si al menos tres vistas *adicionales* contienen algún vecino a no más de $r = 6$ mm:

$$s_i(\mathbf{p}) = \sum_{j \neq i} \mathbf{1}\left[\min_{\mathbf{q} \in P_j} \|\mathbf{p} - \mathbf{q}\|_2 \leq r\right], \qquad s_i(\mathbf{p}) \geq 3, \quad r = 6\,\text{mm}. \tag{5}$$

Buscamos vecinos con `cKDTree` por vista. El radio equivale a dos vóxeles y tolera pequeñas diferencias debidas a disparidad y pose. Exigir tres vistas *adicionales* implica al menos cuatro capturas independientes, incluida la de origen; elegimos ese requisito al comparar visualmente umbrales de uno, dos y tres.

Fusionamos los puntos respaldados y aplicamos *Statistical Outlier Removal* (SOR). Para cada punto, el filtro promedia las distancias a sus 20 vecinos más cercanos y descarta los que cumplen $\bar{d}_i > \mu_{\bar{d}} + 1{,}5\sigma_{\bar{d}}$, donde $\mu_{\bar{d}}$ y $\sigma_{\bar{d}}$ son la media y desviación estándar de esas distancias en toda la nube. Los 20 vecinos fijan la escala local del análisis; el factor 1,5 resultó más estricto que el 2,0 explorado inicialmente y se eligió por la limpieza visual conseguida, no mediante una optimización estadística. Finalmente volvimos a voxelizar a 3 mm para reducir observaciones redundantes y exportamos los puntos con sus colores a `shoe_final_dl.ply` [7].

Medición

Estimamos los ejes longitudinal y transversal mediante PCA sobre el plano XY de la nube final. Para evitar que unos pocos puntos extremos dominen las dimensiones, utilizamos los percentiles 1 y 99 sobre las coordenadas proyectadas X', Y'; la altura toma como base el plano conocido del tablero:

$$L = q_{99}(X') - q_1(X'), \quad A = q_{99}(Y') - q_1(Y'), \quad H = q_{99}(Z_w). \tag{6}$$

La altura supone que el zapato está apoyado sobre $Z_w = 0$, ya que el recorte elimina puntos cercanos a su base. También examinamos la sensibilidad a recortes de 0,5 %, 1 % y 2 %.

Resultados y Discusión

Calibración y rectificación

Al recalibrar con los 14 pares obtuvimos un RMS de reproyección de 0,474 px y un *baseline* de 59,50 mm, frente a los 59,46 mm guardados. La mediana del RMS epipolar vertical fue 0,34 px entre las 26 capturas con correspondencias disponibles: la rectificación alineó las imágenes, sin que esto asegure una profundidad exacta. La tabla 1 resume la calibración utilizada.

La rotación y traslación relativas de la cámara izquierda a la derecha, obtenidas durante la calibración, fueron $\mathbf{r} = (-0{,}00236,\ 0{,}01434,\ 0{,}00117)$ rad (en representación de Rodrigues) y $\mathbf{t} = (-59{,}463,\ -0{,}028,\ 0{,}312)$ mm. La traslación es predominantemente horizontal y su norma corresponde al *baseline* de 59,46 mm.

Tabla 1: Parámetros intrínsecos (px) y de distorsión (adimensionales) de ambas cámaras.

	Izquierda	**Derecha**
$f_x,\ f_y$	403,89; 402,61	404,33; 402,96
$c_x,\ c_y$	649,16; 363,00	634,81; 355,28
k_1, k_2, k_3	0,00912; −0,02647; 0,00434	0,00924; −0,02582; 0,00412
$p_1,\ p_2$	0,00085; 0,00019	0,00054; −0,00023

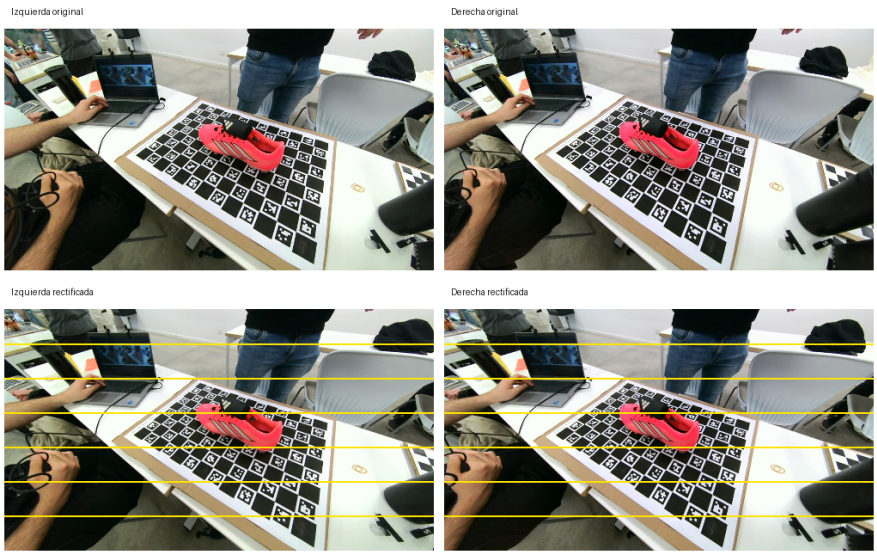
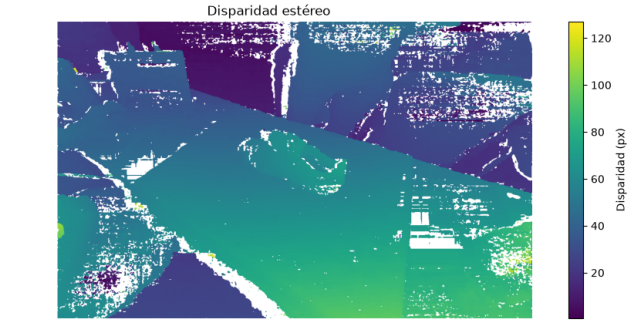


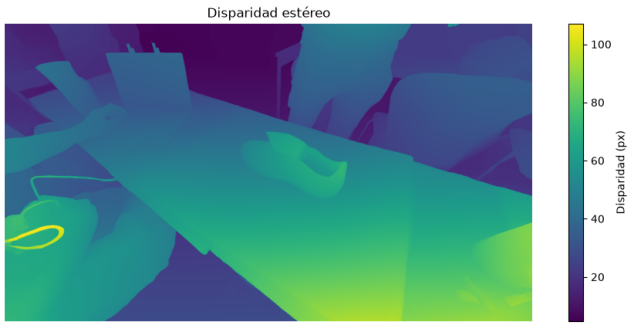
Figura 1: Captura 0 antes y después de rectificar; líneas horizontales como guías visuales.

Disparidad y poses

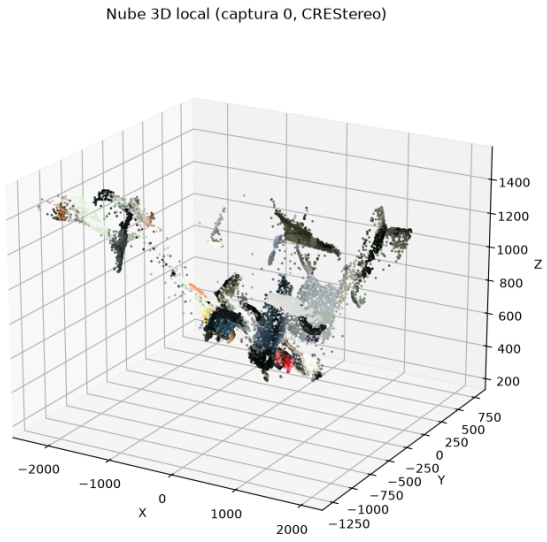
En la captura 0, SGBM presenta regiones inválidas y discontinuidades; CREStereo recupera un mapa visualmente más continuo (figura 2). La nube local todavía incluye el tablero y el entorno, de ahí la necesidad de transformar y recortar las observaciones. Los mapas se muestran con escalas cromáticas independientes y no permiten, por sí solos, comparar errores métricos.



(a) StereoSGBM



(b) CREStereo



(c) Nube local CREStereo

Figura 2: Mapas de disparidad de la captura 0 y nube tridimensional local reconstruida con CREStereo. Las escalas cromáticas son independientes.

Recuperamos **27 poses de 28 capturas**; la única sin pose válida fue la 26. La mediana del RMSE PnP fue 1,47 px. Como comprobación tridimensional adicional, evaluamos la distancia $|Z_w|$ entre las esquinas ChArUco reproyectadas y el plano conocido $Z_w = 0$: la mediana entre las medianas por captura fue de 2,30 mm. Este diagnóstico se calculó con disparidades de **SGBM**, no de CREStereo; algunas tomas presentaron errores bastante mayores. Por lo tanto, sirve para evaluar la coherencia geométrica sobre el tablero, no para asignarle una precisión de 2,30 mm al zapato.

Las posiciones y orientaciones estimadas rodean el tablero y apuntan hacia la zona del objeto, como se espera del recorrido de captura (figura 3). Esta verificación es visual: sin posiciones físicas de referencia, no permite medir el error de las cámaras en el mundo.

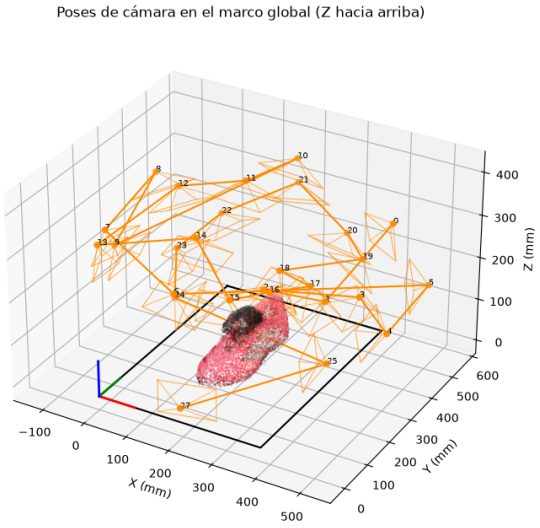


Figura 3: Poses válidas y nube CREStereo en el marco global ChArUco.

Fusión y optimización

En SGBM/D (21 vistas) probamos SOR global con DBSCAN (35 783 puntos), filtrado local con DBSCAN (34 556) y soporte de tres vistas adicionales (29 009 tras voxelizar). Este último dejó menos agrupaciones espurias en la inspección visual, por lo que mantuvimos su criterio con CREStereo. La tabla 2 muestra los recuentos definitivos para las 27 vistas.

Tabla 2: Puntos conservados por etapa con CREStereo; los recuentos previos al último vóxel incluyen observaciones de múltiples vistas.

Etapa	**Puntos**
ROI del objeto (suma de vistas ya voxelizadas)	144 167
Soporte de 3 vistas adicionales, $r = 6$ mm	135 961
SOR (20 vecinos; 1,5 desviaciones)	126 951
Voxelización final, 3 mm	24 037

Los recuentos parten de la *suma de nubes ya recortadas y voxelizadas por vista*; no registramos por separado los puntos anteriores a esas dos operaciones. Entre el recorte y SOR todavía hay varias observaciones sobre una misma superficie, por lo que la reducción de 126 951 a 24 037 puntos al voxelizar elimina sobre todo redundancia, no una proporción equivalente de superficie. La figura 4 muestra cómo el soporte entre vistas y SOR reducen puntos aislados, aunque ambos pueden quitar detalles válidos donde hay poca cobertura.



Figura 4: ROI, soporte multivista y nube optimizada, con el mismo encuadre.

Medición y discusión

Obtuvimos 283,6 × 92,7 × 114,0 **mm** (largo, ancho y alto) mediante PCA, percentiles y altura desde el tablero (figura 5). Con SGBM/D habíamos medido 284,6 × 96,7 × 114,3 mm. Las diferencias son pequeñas en largo y alto, pero alcanzan unos 4 mm en ancho. Sin medidas físicas independientes, esta proximidad no demuestra exactitud métrica.

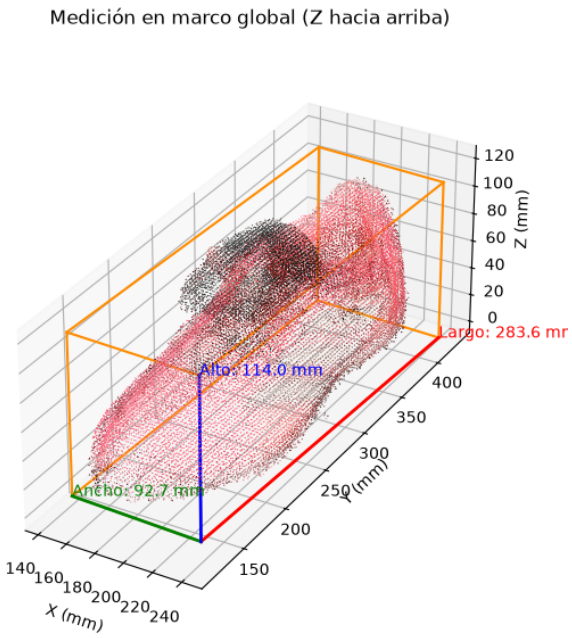


Figura 5: Nube final de 24 037 puntos y caja de medición en milímetros.

CREStereo dio la mejor reconstrucción *visual* de las variantes probadas, pero también cambiamos la cantidad de vistas (de 21 a 27) y el filtrado; no podemos atribuirle toda la mejora al estimador neuronal. La consistencia multivista ayudó a descartar puntos sin respaldo entre capturas, mientras que SOR atacó el ruido residual sin garantizar que cada punto conservado sea correcto. Persisten limitaciones por oclusiones, textura insuficiente, ROI manual y el supuesto de apoyo sobre el plano ChArUco. Una evaluación de exactitud requeriría medir el zapato físicamente; para aislar el efecto de cada decisión habría que comparar ambos estimadores con las mismas vistas y filtros.

Referencias

[1] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2004.

[2] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.

[3] Heiko Hirschmüller. Stereo processing by semiglobal matching and mutual information. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(2):328–341, 2008.

[4] Jiankun Li, Peisen Wang, Pengfei Xiong, Tao Cai, Ziwei Yan, Lei Yang, Jiangyu Liu, Haoqiang Fan, and Shuaicheng Liu. Practical stereo matching via cascaded recurrent network with adaptive correlation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16263–16272, 2022.

[5] OpenCV. Camera calibration and 3d reconstruction: calib3d module. OpenCV documentation. Documentación oficial, consultada en octubre de 2026.

[6] OpenCV. Detection of charuco boards. OpenCV tutorials. Documentación oficial, consultada en octubre de 2026.

[7] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3D: A modern library for 3D data processing. *arXiv:1801.09847*, 2018.